Stanescu Miruna-Ioana
334AA

Gestionarea retetelor culinare

Aplcatie Web pentru gestionarea retetetelor culinare.

Cuprins

1. Descrierea proiectului
   1.1 Functionalitate.....
   1.2 Motivatie.....
   1.3 Medii de lucru.....
   1.4 Structura proiectului.....

2. Baza de date
   2.1 Structura bazei de date.....
   2.2 Clase de baza.....

3. Aplicatie Web
   3.1 Pagina de start.....
   3.2 Pagina de retete.....
   3.3 Pagina de adaugare.....
   3.4 Pagina de modificare.....
   3.5 Pagina de cautare.....

4. Metode
   4.1 Setters & Getters.....
   4.2 Constructori.....
   4.3 RetetaService.....
   4.4 IngredientService, EtapaService, UstensilaService.....

5. Validare
   5.1 Validare la adaugarea retetei.....
   5.2 Validare pentru modificarea si cautarea retetelor.....

6. Testare
   6.1 Testare Setters & Getters.....
   6.2 Testare metode Service.....

7. Concluzie
   7.1 Imbunatatirea aplicatiei.....
   7.2 Concluzie.....

Stanescu Miruna-Ioana
334AA

1. Descrierea proiectului

1.1 Functionalitate

Proiectul are ca scop gestionarea retetelor culinare personale pentru a usura munca oamenilor in bucatarie. Prin prisma acestei aplicatii, utilizatorul isi poate salva retetele ca mai apo isa le poata cauta in functie de ingredientele pe care le are in casa. Utilizatorul isi poate adauga retetele, le poate modifica, le poate sterge si vizualiza pe toate intr-un formar atragator si aspetcuos. Totodata, acesta poate vedea in functie de ingredientele pe care le are acasa care este cea mai usor de realizat reteta.

1.2 Motivatie

Am ales aceasta tema deoarece, munca in bucatarie este una destul de grea, iar ideile pentru fiecare masa nu vin atat de usor. De accea, consider ca o aplicatie pentru gestionarea retetelor poate fi foarte utila si folosita in viata de zi cu zi, mai ales ca poate scuti omul de niste cheltuieli suplimentare prin funtionalitatea adaugata de cautare a retetelor.

1.3 Medii de lucru

Implementarea acestui proiect a fost facuta cu ajutorul limajului de programare Java pentru partea de back-end, iar pentru partea de front-end am folosit HTML pentru template-urile paginilor si CSS pentru stilizarea acestora. Ca platforma de lucur am utilizat InteliJ, proiectul a fost create cu Spring Initializr, cu optiunile Maven, Spring Boot versiunea 3.3.7, iar ca baza de date am folosit H2-database.

1.4 Structura proiectului

Proiectul retete-culinare are, printer multe altele, urmatoarele foldere main(java, resources) si test:

In java se gasesc:

- Controller (unde este scrisa logica functiilor apelate in HTML)
- Model (continue modelele componentelor din baza de date)
- Repository si Service (functiile fiecarei entitati, care mai apoi sunt apelate in controller)

- ReteteCulinareApplication (fisier care prin rulare porneste aplicatia)

In resources se gasesc:

- Static (imagini si cod CSS pentru stilizare)
- Templates (fisiere HTML pentru fiacare pagina in parte)
- Application.properties (fisier pentru gestionarea bazei de date)

In test se gasesc clasele in care sunt testate functiile fiecarei entitati.

2. Baza de date

2.1 Structura bazei de date

Pentru a salva datele, am folosit baza de date h2-database. Proiectul retete culinare necesita urmatoarele tabele si campuri: Reteta, Ingredient, Etapa, Ustensila.

Ingredientele, Etapele si Ustensilele au un camp in tabela pentru RetetaID, astfel incat acestea sa fie associate unei singure retete, chiar daca poate au acelasi nume, acestea pot avea de exemplu cantitati diferite sau timpi diferiti.

In tabela Reteta sunt salvate numele, categoria, costul si timpul de preparare pentru fiecare reteta in parte.

ID	CATEGORIE	COST	NUME	TIMP_PREPARARE
12	Mic-dejun	4.695	Omeleta cu legume	10
13	Pranz	5.069999999999999	Salata de quinoa cu legume	20
18	Aperitiv	8.75	Salata de ton	9
19	Pranz	6.4	Supa crema de rosii	22
20	Cina	18.9	Pui cu sos de soia	30
21	Cina	33.85	Lasagna Bolognese	95
22	Pranz	23.65	Risotto cu ciuperci	35
23	Pranz	17.100000000000005	Salata de pui cu avocado	28
24	Aperitiv	16.735000000000003	Goacamole	13
25	Cina	10.415	Somon cu legume la cuptor	48
26	Desert	7.05	Brownie cu nuci	45
27	Mic-dejun	2.6399999999999997	Porridge cu fructe de padure	9
28	Pranz	13.440000000000001	Tocanita de vita cu cartofi	65
29	Aperitiv	6.109999999999999	Humus cu paine pita	41
30	Desert	7.17	Clatite cu ciocolata	38
31	Cina	41.36	Muschi de vita cu sparanghel	45

Stanescu Miruna-Ioana
334AA

In tabela ingredient sunt salvate numele, categoria, cantitatea, unitatea de masura, pretul per unitate, pretul pe cantitate si id-ul retetei.

ID	CANTITATE	CATEGORIE	NUME	PRET_PRODUS	PRET_TOTAL	UNITATE_MASURA	RETETA_ID
7	450.0	Lactate	Oua	6.0	2.7	Grame	12
8	400.0	Legume	Rosii	3.0	1.2000000000000002	Grame	12
9	100.0	Legume	Ardei gras	4.0	0.4	Grame	12
10	10.0	Uleiuri si grasimi	Ulei de masline	30.0	0.3	Mililitri	12
11	10.0	Condimente	Sare	2.0	0.02	Grame	12
12	5.0	Condimente	Piper	15.0	0.075	Grame	12
13	200.0	Cereale	Quinoa	15.0	3.0	Grame	13
14	150.0	Legume	Castravate	2.0	0.3	Grame	13
15	400.0	Legume	Rosii	3.0	1.2000000000000002	Grame	13
16	10.0	Uleiuri si grasimi	Ulei de masline	30.0	0.3	Mililitri	13
17	50.0	Fructe	Lamaie	5.0	0.25	Grame	13
18	10.0	Condimente	Sare	2.0	0.02	Grame	13
20	200.0	Peste	Ton conservat	30.0	6.0	Grame	18
21	50.0	Sosuri	Maioneza	40.0	2.0	Grame	18
22	150.0	Legume	Castravate	5.0	0.75	Grame	18
23	500.0	Legume	Rosii	6.0	3.0	Grame	19
24	100.0	Legume	Ceapa	4.0	0.4	Grame	19
25	100.0	Lactate	Smantana	30.0	3.0	Grame	19
26	500.0	Carne	Piept de pui	35.0	17.5	Grame	20
27	50.0	Sosuri	Sos de soia	20.0	1.0	Mililitri	20
28	10.0	Legume	Usturoi	40.0	0.4	Grame	20
29	250.0	Panficatie	Foi de lasagna	15.0	3.75	Grame	21
30	500.0	Carne	Carne tocata de vita	40.0	20.0	Grame	21
31	100.0	Legume	Ceapa	4.0	0.4	Grame	21
32	400.0	Legume	Rosii pasate	7.0	2.8000000000000003	Grame	21
33	200.0	Lactate	Mozzarella	30.0	6.0	Grame	21
34	30.0	Uleiuri si grasimi	Ulei de masline	25.0	0.75	Mililitri	21
35	10.0	Condimente	Sare si piper	15.0	0.15	Grame	21

In tabela etapa sunt salvate numele, timpul de preparare, descrierea si id-ul retetei.

SELECT * FROM ETAPA;

ID	DESCRIERE	NUME	TIMP	RETETA_ID
4	Bate ouale intr-un bol cu ajutorul unui tel sau furculite pana devin omogene.	Batut ouale	2	12
5	Taie rosiile si ardeiul in cuburi mici pentru a fi adaugate la omeleta.	Taiat legumele	2	12
6	Intr-o tigaie, incalzeste uleiul de masline si adauga legumele taiate. Caleste-le la foc mediu pana se inmoaie.	Calit legumele	3	12
7	Toarna ouale batute peste legumele calite si lasa-le sa se coaca la foc mic, fara a amesteca, pana cand ouale sunt complet gatite.	Adaugat ouale batute	2	12
8	Serveste omeleta imediat, presarand sare si piper dupa gust.	Servire	1	12
9	Fierbe quinoa conform instructiunilor de pe ambalaj si lasa-o sa se raceasca.	Fiert quinoa	10	13
10	Taie castravetele si rosiile in bucati mici si pune-le intr-un bol.	Taiat legumele	5	13
11	Adauga quinoa fiarta si racita peste legumele taiate si amesteca bine.	Amestecat quinoa cu legumele	2	13
12	Stoarce sucul de lamaie si adauga uleiul de masline. Amesteca totul bine.	Adaugat ulei de masline si lamaie	2	13
13	Serveste salata imediat, presarand sare dupa gust.	Servire	1	13
15	Deschide conserva și scurge lichidul.	Deschide tonul	1	18
16	Taie castraveți cubulețe.	Taierea castravetilor	3	18
17	Combină tonul cu maioneza și castraveții.	Amestecarea ingredientelor	5	18
18	Gătește ceapa în ulei până devine transparentă.	Soteaza ceapa	5	19
19	Adaugă roșiile tăiate și lasă să fiarbă.	Adauga rosiile	10	19
20	Folosește un blender pentru a obține o consistență fină.	Mixeaza supa	5	19
21	Incorporează smântâna și servește.	Adauga smantana	2	19
22	Taie pieptul de pui în cuburi.	Taiera puiului	5	20
23	Amestecă puiul cu sosul de soia și usturoiul zdrobit.	Marinarea puiului	10	20
24	Prăjește puiul într-o tigaie până devine auriu.	Prajirea puiului	15	20
25	Într-o tigaie, adaugă ceapa tăiată și călește până devine aurie.	Soteaza ceapa	5	21
26	Adaugă carnea tocată și gătește până se rumenește.	Adauga carnea	10	21
27	Încorporează roșiile pasate, sarea și piperul și lasă să fiarbă 20 de minute.	Adauga rosiile	20	21
28	Într-un vas de copt, alternează straturi de foi de lasagna, carne și mozzarella.	Asambleaza lasagna	15	21
29	Coace la cuptor la 180°C până se rumenește.	Coace lasagna	45	21
30	Gătește ceapa în ulei de măsline până devine translucidă.	Soteaza ceapa	5	22
31	Adaugă ciupercile tăiate și gătește până scade apa.	Adauga ciupercile	5	22
32	Incorporează orezul și lasă-l să absoarbă aromele.	Adauga orezul	2	22
33	Adaugă supa de pui treptat, amestecând constant, până orezul este gătit.	Adauga supa treptat	20	22

In tabela ustensila sunt salvate numele, categoria si id-ul retetei.

ID	CATEGORIE	NUME	RETETA_ID
6	Vase	Tigaie	12
7	Instrumente	Cutit	12
8	Recipiente	Bol	12
9	Ustensile	Tel	12
10	Ustensile	Tocator	12
11	Vase	Oala	13
12	Instrumente	Cutit	13
13	Instrumente	Lingura	13
14	Ustensile	Tocator	13
16	Vase	Bol	18
17	Instrumente	Cutit	18
18	Ustensile	Lingura	18
19	Vase	Oala	19
20	Electrocasnice	Blender	19
21	Ustensile	Lingura	19
22	Ustensile	Tigaie	20
23	Vase	Bol	20
24	Instrumente	Spatula	20
25	Ustensile	Tigaie	21
26	Recipiente	Vas de copt	21
27	Instrumente	Cutit	21
28	Vase	Oala	22
29	Instrumente	Lingura de lemn	22
30	Instrumente	Rasnita pentru branza	22
31	Ustensile	Tigaie	23

2.2 Clase de baza

Pentru a avea acces la aceste date in InteliJ au fost create clase pentru fiecare tabela in parte cu parametrii asociati fiecareia.

Reteta:

```java
@Entity
public class Reteta {
    1 usage
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    3 usages
    private String nume;
    2 usages
    private Double cost = 0.0;
    2 usages
    private Integer timpPreparare = 0;
    3 usages
    private String categorie;
    1 usage
    @OneToMany(mappedBy = "reteta", cascade = CascadeType.ALL)
    private List<Ingredient> ingrediente;
    1 usage
    @OneToMany(mappedBy = "reteta", cascade = CascadeType.ALL)
    private List<Etapa> etape;
    1 usage
    @OneToMany(mappedBy = "reteta", cascade = CascadeType.ALL)
    private List<Ustensila> ustensile;
```

Pe langa parametrii uzuali ai clasei reteta din baza de date, mai trebuie adaugati si cei de tip lista pentru fiecare dintre Ingredient, Etapa si Ustensila pentru a defini relatia dintre tabele one-to-many.

Ingredient:

```
@Entity
public class Ingredient {
    2 usages
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    2 usages
    @ManyToOne
    @JoinColumn(name = "reteta_id", nullable = false)
    private Reteta reteta;
    2 usages
    private String nume;
    2 usages
    private String categorie;
    2 usages
    private Double cantitate;
    2 usages
    private String unitateMasura;
    2 usages
    private Double pretProdus;
    2 usages
    private Double pretTotal;
```

A fost adaugata in definitie relatie de legatura many-to-one cu tabela reteta. Asemanator cu clasa pentru Ingredient sunt create si clasele pentru Etapa si Ustensila:

```
@Entity
public class Etapa {
    2 usages
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    2 usages
    @ManyToOne
    @JoinColumn(name = "reteta_id", nullable = false)
    private Reteta reteta;
    2 usages
    private String nume;
    2 usages
    private String timp;
    2 usages
    private String descriere;
```

```
@Entity
public class Ustensila {
    2 usages
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    1 usage
    @ManyToOne
    @JoinColumn(name = "reteta_id")
    private Reteta reteta;
    2 usages
    private String nume;
    2 usages
    private String categorie;
```

3. Aplicatia web

3.1 Pagina de start

Utilizatorul vede pentru inceput o pagina eleganta de start, cu titlu, niste imagini si o scurta descriere. Din aceasta pagina, el poate sa vada retetele, sa adauge unele noi sau sa le caute.

Pe parte de cod, pentru aceasta pagina nu exista foarte multe functii create. Prin codul HTML al acesteia, utilizatorul este redirectionat pe pagina de retete daca apasa butonul de Retete sau Vezi Retele, pe pagina de Adauga Reteta sau pe pagina de Cauta Reteta.

3.2 Pagina de retete

Pagina de reteta are mai multe functionalitati, prima este aceeasi in care poti vedea in detaliu fiecare reteta, apoi poti sterge o reteta sau poti modifica o reteta.

- Titlu, descriere si navbar:

- Reteta:

Voi lua pe rand fiecare functionalitate a acestui card pentru o reteta.

1. Cost

Costul este calculat ca suma a fiecarui pret de la fiecare ingredient, iar acest lucru se face prin urmatoarea bucata de cod:

```
List<Ingredient> ingrediente = ingredientService.gasesteIngredienteDupaRetetaId(retetaId);
double costTotal = 0;
for (Ingredient ing : ingrediente) {
    costTotal += ing.getPretTotal();
}
reteta.setCost(costTotal);
retetaService.salveazaReteta(reteta);
```

2. Timp de preparare

Timpul de preparare este calculat ca suma a timpilor de preparare pentru fiecare etapa in parte, prin urmatoarea secventa de cod:

```
etapaService.salveazaEtapa(etapa);

List<Etapa> etape = etapaService.gasesteEtapeDupaRetetaId(retetaId);
int timpTotal = 0;
for (Etapa e : etape) {
    timpTotal += Integer.parseInt(e.getTimp());
}

reteta.setTimpPreparare(timpTotal);
retetaService.salveazaReteta(reteta);
```

3. Afisarea Ingredientelor, a Etapelor si a Ustensilelor

Prin aceasta functie, in codul HTML pot avea acces la toate detaliile retetei si astfel le pot afisa.

```
@GetMapping("/retete")
public String showRetete(Model model) {
    List<Reteta> retete = retetaService.getAllRetete();
    model.addAttribute( attributeName: "retete", retete);
    return "retete";
}
```

4. Sterge Reteta

Cand butonul sterge reteta este apasat, utilizatorul este redirectionat catre o pagina in care trebuie sa confirme stergerea acelei retete:

In raport cu partea de cod, pentru aceasta functie am create un nou template HTML pentru noua pagina de confirmare a stergerii reteti, in care este redirectionat Id-ul retetei, pentru a gasi in baza de date toate detaliile acesteia(ingredient, etape, ustensile) care sunt sterse in cascada.

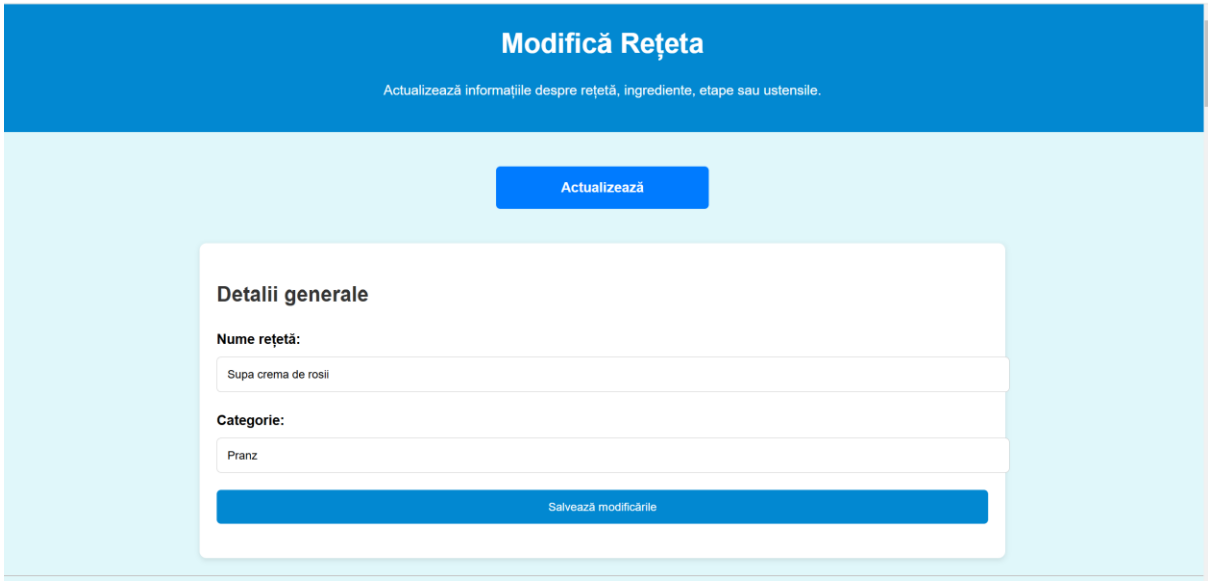
Cod:

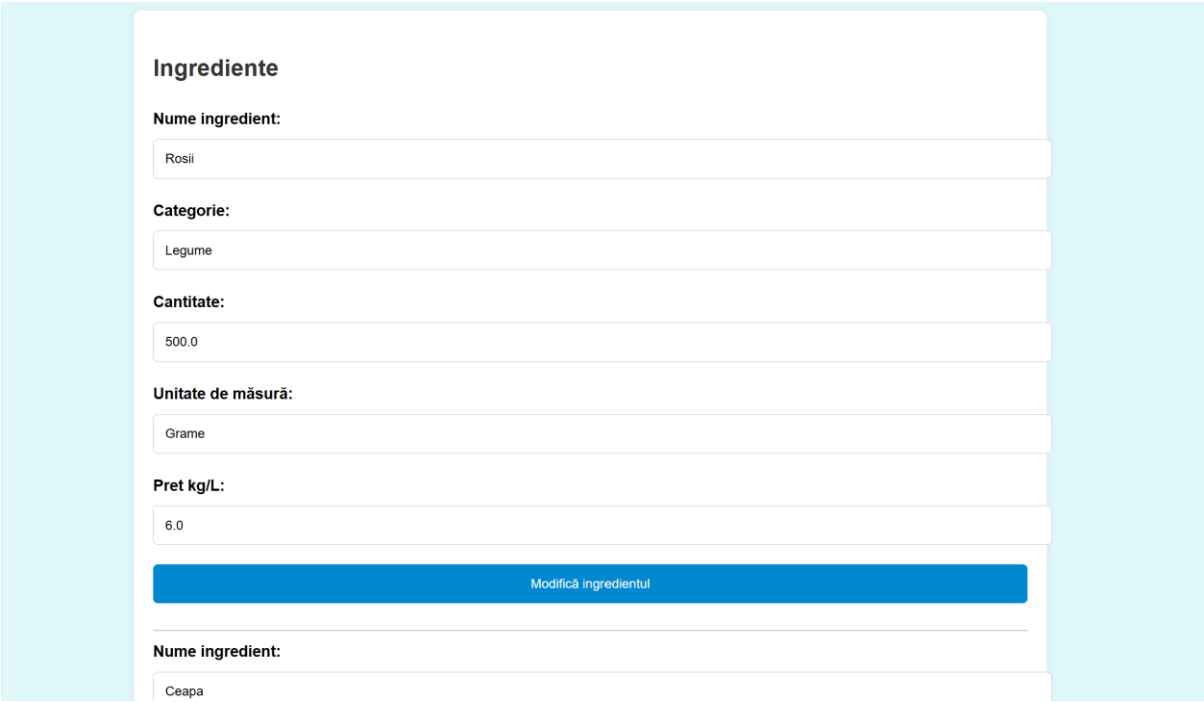
```
no usages
@GetMapping("/stergere-reteta/{id}")
public String confirmDelete(@PathVariable Long id, Model model) {
    Reteta reteta = retetaService.gasesteRetetaDupaId(id);
    model.addAttribute( attributeName: "reteta", reteta);
    return "stergere-reteta";
}

no usages
@PostMapping("/stergere-reteta/{id}")
public String deleteRecipe(@PathVariable Long id) {
    retetaService.stergeReteta(id);
    return "redirect:/retete";
}
```

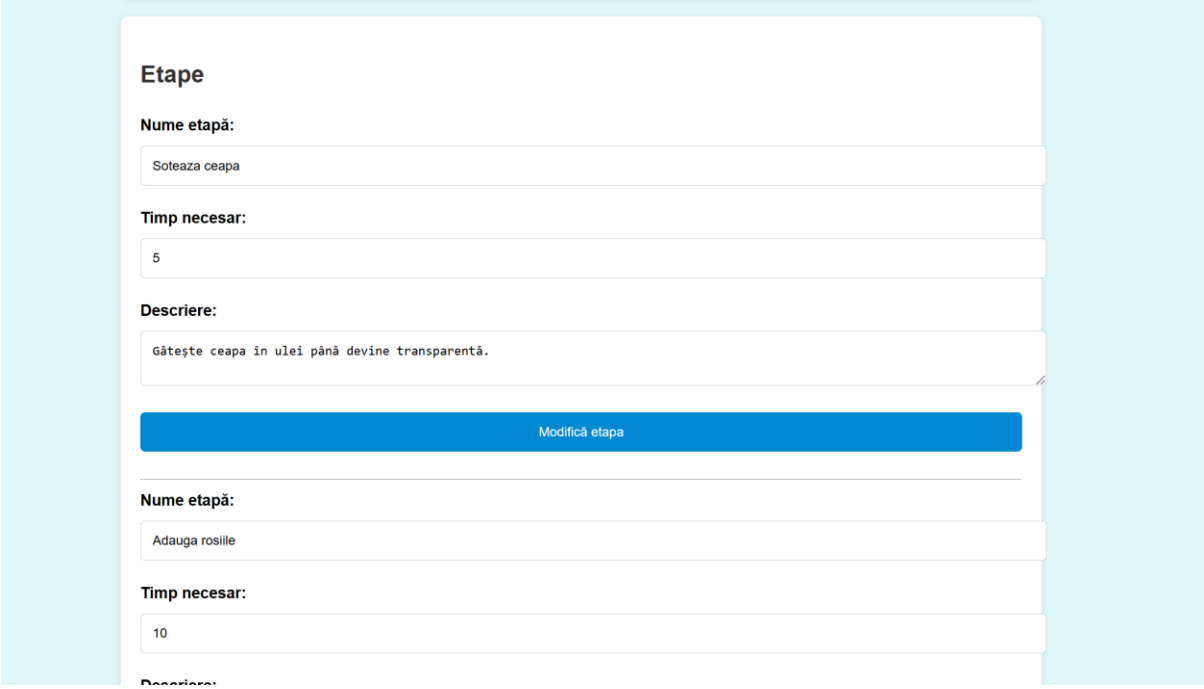
Reteta este cautata dupa id, care este trimis in url-ul pagnii, iar apoi este apelata functia de stergere pe care o voi detalia le sectiunea de metode.

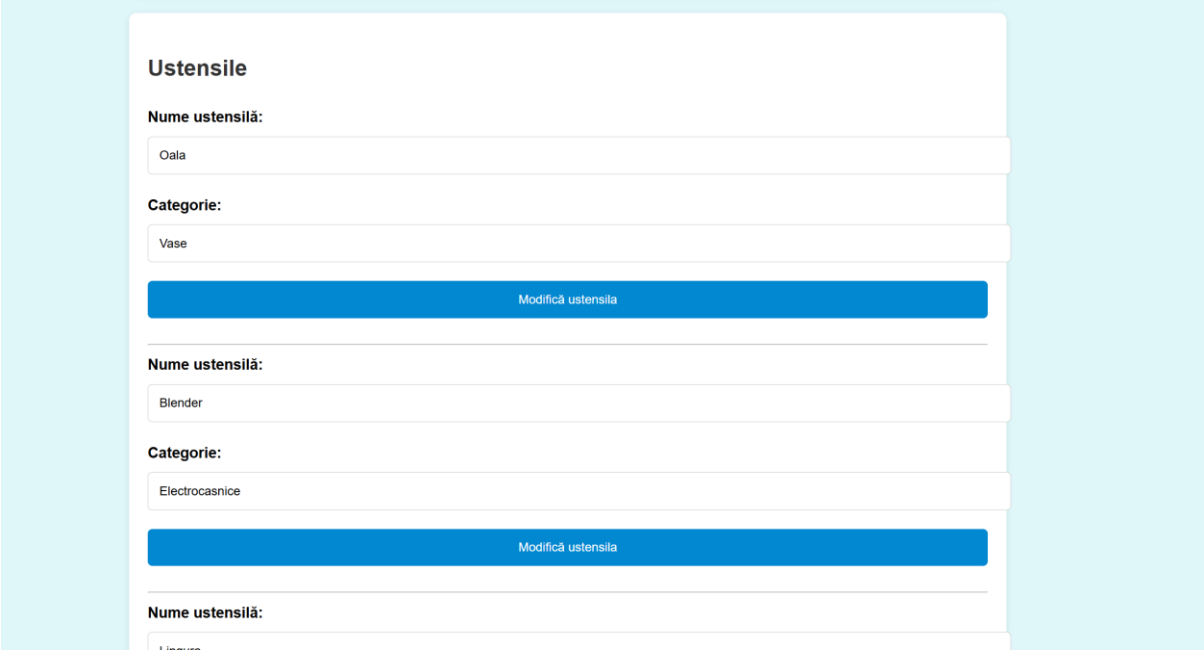
5. Modifica Reteta

Cand utilizatorul apasa butonul de modificare al retetei acesta este redirectionat catre o pagina in care poate modifica fiecare camp in parte al retetei, inclusive fiecare camp in parte al fiecarui ingredient, fiecarei etape si fiecarei ustensile.



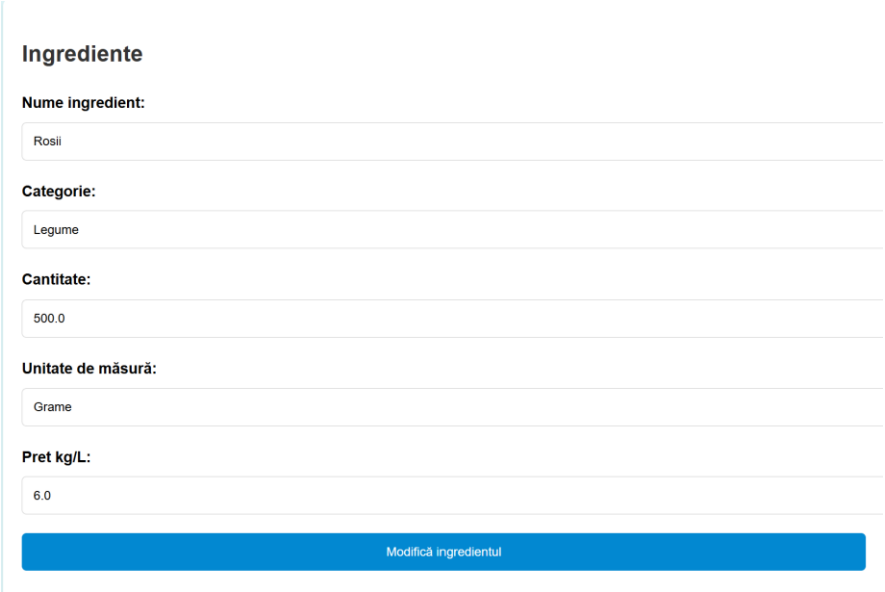




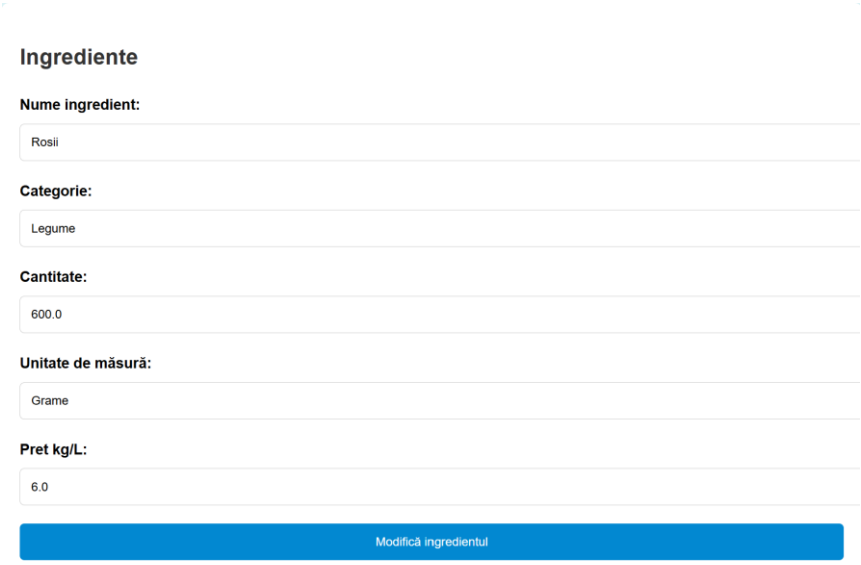


Pentru a testa aceasta functionalitate vom incerca sa modificam cantitatea de rosii pe care o punem la supa. De asemenea, daca utilizatorul pune valori invalide, aplicatia nu ii va permite si va primi un mesaj de eroare cu detalii despre cum arata niste date valide(vom detalia la etapa de validare).

Inainte:



Dupa:



Pentru a putea implementa aceasta functionalitate am folosit mai multe functii de modificare pentru fiecare camp in parte.

Pentru a pastra valorile deja existente ale retetei am adaugat in model atributele ingredientelor, ale etapelor, ale ustensilelor si campurile retetei, iar pentru modificare am utilizat setteri si functia de update.

```
Reteta reteta = retetaService.gasesteRetetaDupaId(id);
reteta.setNume(nume);
reteta.setCategorie(categorie);

retetaService.updateReteta(id, reteta);
```

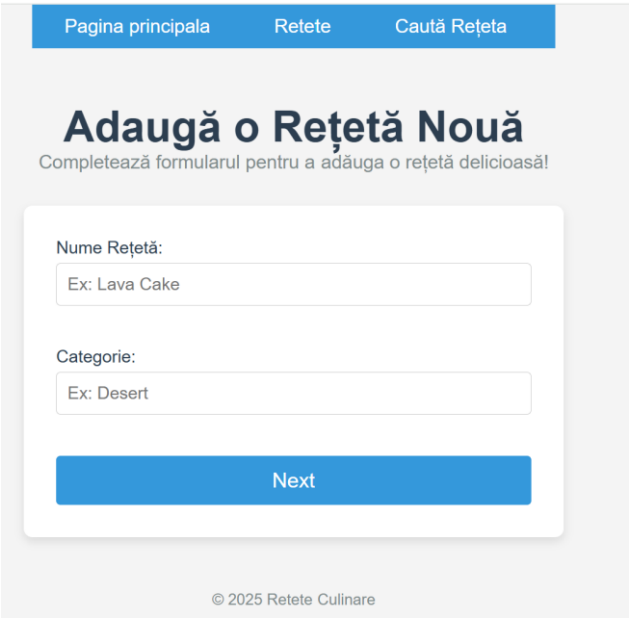
La modificarea ingredientelor si a etapelor si actualizeaza automat costul total si timpul de preparare astfel:

```
double pretTotal;
if(!unitateMasura.matches( regex: "Bucata"))
    pretTotal = cantitateValoare * 0.001 * pretProdusValoare;
else
    pretTotal = cantitateValoare * pretProdusValoare;

List<Ingredient> ingrediente = ingredientService.gasesteIngredienteDupaRetetaId(idReteta);
double costTotal = 0;
for (Ingredient ing : ingrediente) {
    costTotal += ing.getPretTotal();
}
reteta.setCost(costTotal);
retetaService.salveazaReteta(reteta);

Ingredient ingredient = ingredientService.gasesteDupaId(id);
ingredient.setNume(nume);
ingredient.setCategorie(categorie);
ingredient.setCantitate(cantitateValoare);
ingredient.setUnitateMasura(unitateMasura);
ingredient.setPretProdus(pretProdusValoare);
ingredient.setPretTotal(pretTotal);
ingredientService.updateIngredient(id, ingredient);
```

```
Etapa etapa = etapaService.gasesteDupaId(id);
etapa.setNume(nume);
etapa.setTimp(timp);
etapa.setDescriere(descriere);
etapaService.updateEtapa(id, etapa);

List<Etapa> etape = etapaService.gasesteEtapeDupaRetetaId(idReteta);
int timpTotal = 0;
for (Etapa e : etape) {
    timpTotal += Integer.parseInt(e.getTimp());
}

reteta.setTimpPreparare(timpTotal);
retetaService.salveazaReteta(reteta);
```

3.3 Pagina de adaugare

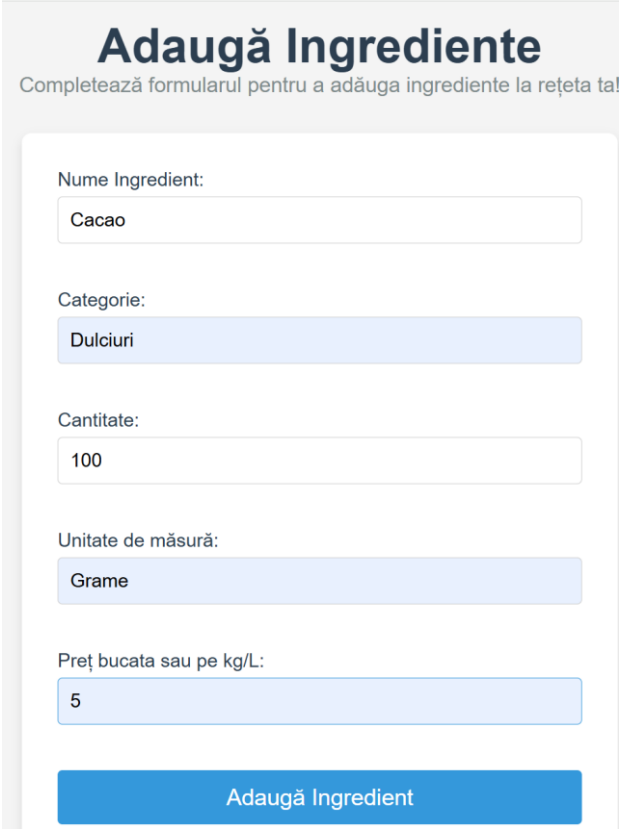
La apasarea butonului de adaugare reteta utilizatorul este redirectionat catre pagina de adaugare reteta, in care poate introduce numele si categoria reteti.

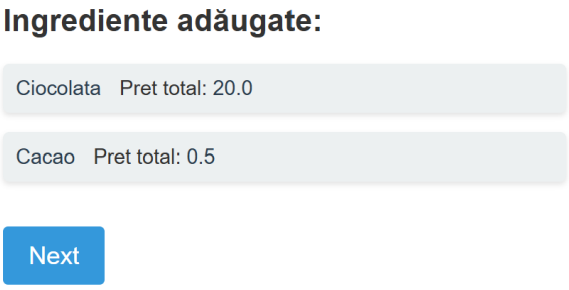


In momentul in care datele valide sunt introduce si butonul de next este apasat se apeleaza fucntia de aduagare reteta prin care se creeaza un nou obiect de tip Reteta, iar apoi utilizatorul este redirectionat catre formularul de adaugare ingrediente:

```
Reteta reteta = new Reteta();
reteta.setNume(nume);
reteta.setCategorie(categorie);

retetaService.salveazaReteta(reteta);

model.addAttribute( attributeName: "retetaId", reteta.getId());

return "redirect:/ingrediente?retetaId=" + reteta.getId();
```

In pagina de adaugare a ingredientelor ultilizatorul poate adauga cate ingredeinete are nevoie pentru reteta prin butonul ingredient, care se vor afisa sub forma de lsiata sub acesta, iar apoi, cand apasa pe butonul de next este redirectionat catre formularul de adaugare etape:



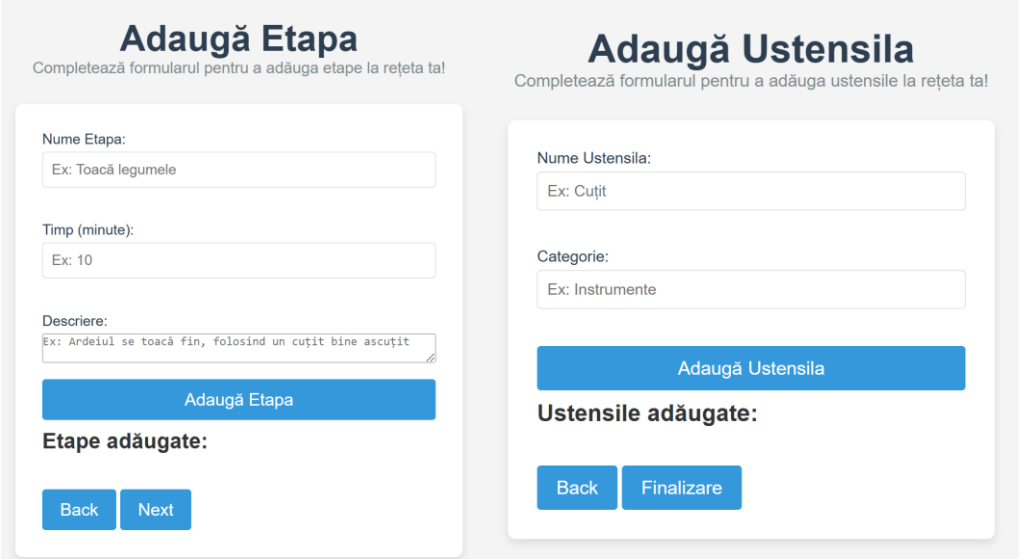


Aici este calculate si pretul total pentru fiecare ingredient in functie de unitatea de masura(Bucata/Kg sau L)

```
    double pretTotal;

if(unitateMasura.matches( regex: "Grame") || unitateMasura.matches( regex: "Mililitri"))
    pretTotal = cantitateValoare * 0.001 * pretProdusValoare;
else
      pretTotal = cantitateValoare * pretProdusValoare;
```

Salvarea datelor pentru fiecare ingredient se face in cod similar ca pentru retea. Se creeaza o noua instanta a clasei Ingredient si datele sunt salvate in baza de date la apsarea butonului Adauga Ingredient.

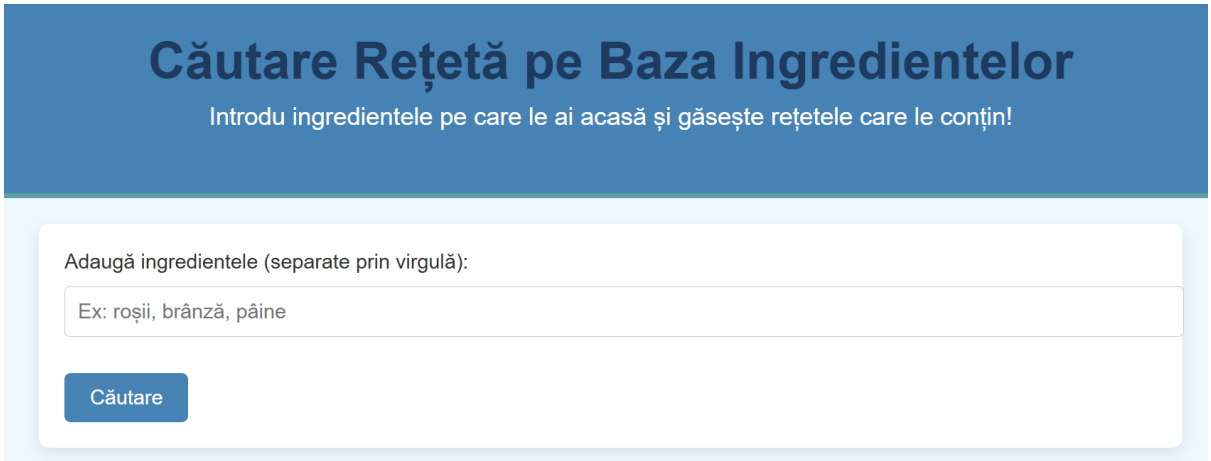
La fel se procedeaza si pentru Etapa si Ustensila



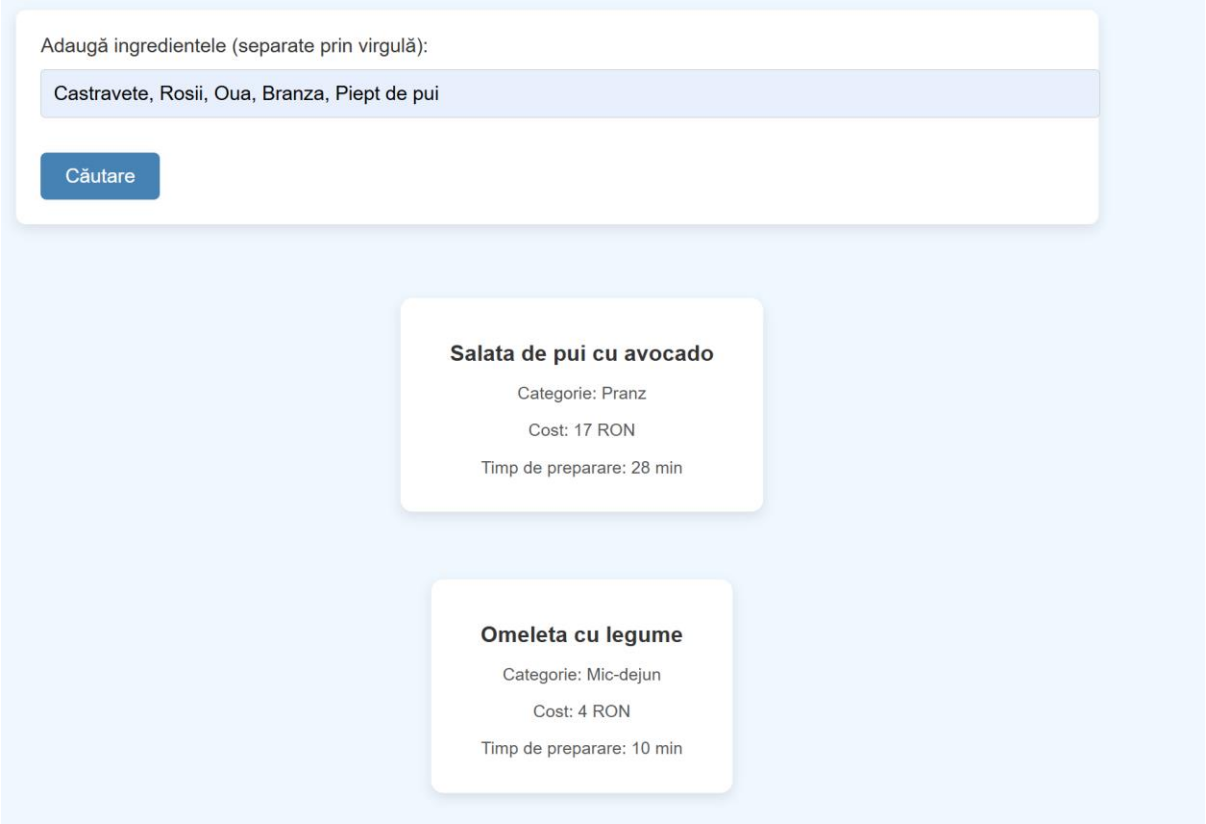
Dupa ce butonul Finalizare este apasat, reteta este salvata in totalitate, iar utilizatorul este redirectionat catre pagina de retete.

3.4 Pagina de Cautare

Aceasta pagina este facuta pentru a permite utilizatorului sa introduca ingredientele pe care le are in casa si sa primeasca cele mai rentabile retete de facut.



Utilizatorul introduce datele si va primi lista astfel:



Iar mai jos este o lista intreaga cu fiecare reteta care continue cel putin un ingredient din cele introduce.

Pentru inceput sirul de caractere este separate in elemente diferite si puse intr o lista cu functia stream:

```
List<String> ingredienteCautate = Arrays.stream(ingrediente.split( regex: ","))
        .map(String::trim)
        .collect(Collectors.toList());
```

Acest lucru s a facut prin functia findAll, care imi da toate retetele care contin ingredientele respective, si apoi se creeaza un obicet de tip HashMap care va asocia retetele cu numarul de ingredient comune:

```
List<Reteta> reteteGasite = retetaService.findAll();
Map<Reteta, Long> reteteCuNumarDeIngrediente = new HashMap<>();
```

Apoi, sunt numerate ingredientele commune, iar retetele sunt sortate dupa acesta si afisate in pagina:

```
for (Reteta reteta : reteteGasite) {
    if (reteta.getIngrediente() != null) {
        long numarDeIngredienteComune = reteta.getIngrediente().stream()
                .filter(ingredient -> ingredienteCautate.contains(ingredient.getNume()))
                .count();
        if (numarDeIngredienteComune > 0) {
            reteteCuNumarDeIngrediente.put(reteta, numarDeIngredienteComune);
        }
    }
}

List<Reteta> reteteSortate = reteteCuNumarDeIngrediente.entrySet().stream() Stream<Entry<...>>
        .sorted((entry1, entry2) -> Long.compare(entry2.getValue(), entry1.getValue()))
        .map(Map.Entry::getKey) Stream<Reteta>
        .collect(Collectors.toList());

model.addAttribute( attributeName: "retete", reteteSortate);
```

4. Metode

4.1 Setters & Getters

Pentru fiecare clasa am facut setters & getters pentru a putea avea acces sau pentru a modifca atributele acestora.

Reteta:

```
public Long getId() { return id; }
public String getNume() { return nume; }
public String getCategorie() { return categorie; }
1 usage
public Double getCost() { return cost; }
3 usages
public void setCost(Double cost) { this.cost = cost; }
3 usages
public void setTimpPreparare(Integer timpPreparare) { this.timpPreparare = timpPreparare; }
1 usage
public Integer getTimpPreparare() { return timpPreparare; }
public void setNume(String nume) { this.nume = nume; }
public void setCategorie(String categorie) { this.categorie = categorie; }

2 usages
public List<Ingredient> getIngrediente() { return ingrediente; }
no usages
public List<Etapa> getEtape() { return etape; }
no usages
public List<Ustensila> getUstensile() { return ustensile; }
```

Asemantror pentru Ingredient, Etapa si Ustensila.

4.2 Constructori

Am facut constructori cu si fara parametri:

```
public Reteta(String nume, String categorie, String imagineUrl) {
    this.nume = nume;
    this.categorie = categorie;
}
public Reteta(){}
```

4.3 RetetaService

```
6 usages
public Reteta salveazaReteta(Reteta reteta) { return retetaRepository.save(reteta); }

2 usages
public List<Reteta> getAllRetete() { return retetaRepository.findAll(); }

16 usages
public Reteta gasesteRetetaDupaId(Long retetaId) { return retetaRepository.findById(retetaId).orElse( other: null); }

2 usages
public void stergeReteta(Long retetaId) { retetaRepository.deleteById(retetaId); }

2 usages
public void updateReteta(Long id, Reteta reteta) { retetaRepository.save(reteta); }
2 usages
public List<Reteta> findAll() {
    String hql = "FROM Reteta";
    Query query = entityManager.createQuery(hql);
    return query.getResultList();
}
```

Functiile de salvare, gaseste dupa id, stergere si update sunt deja predefinte in java prin repository-ul creat, aici acestea sunt doar apelate.

```
package com.retete.retete_culinare.repository;

import com.retete.retete_culinare.model.Reteta;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

4 usages
@Repository
public interface RetetaRepository extends JpaRepository<Reteta, Long> {
}
```

Functia findAll imi creaza un query in baza de date prin care se selecteaza toate retetele respective.

4.4 IngredientService, EtapaService, UstensilaService

Functiile pentru salvare, stergere, cautare dupa id sunt similare, pentru aceste clase am modificat functia de update, folosid setters pentru actualizare:

```
public void updateIngredient(Long id, Ingredient ingredientActualizat) {
    Ingredient ingredientExistent = gasesteDupaId(id);

    ingredientExistent.setNume(ingredientActualizat.getNume());
    ingredientExistent.setCategorie(ingredientActualizat.getCategorie());
    ingredientExistent.setCantitate(ingredientActualizat.getCantitate());
    ingredientExistent.setUnitateMasura(ingredientActualizat.getUnitateMasura());

    ingredientRepository.save(ingredientExistent);
```

La fel si pentru Etapa si Ustensila.

5. Validare

5.1. Validare la adaugarea retetei

Pentru valdiarea la adaugarea unei retete am nevoie se verfic daca numele retetei continue doar litere mari sau mici, continue cel putin o litera si permite utilizarea spatiilor. Astfel, am folosit o expresie regulata pentru verficare:

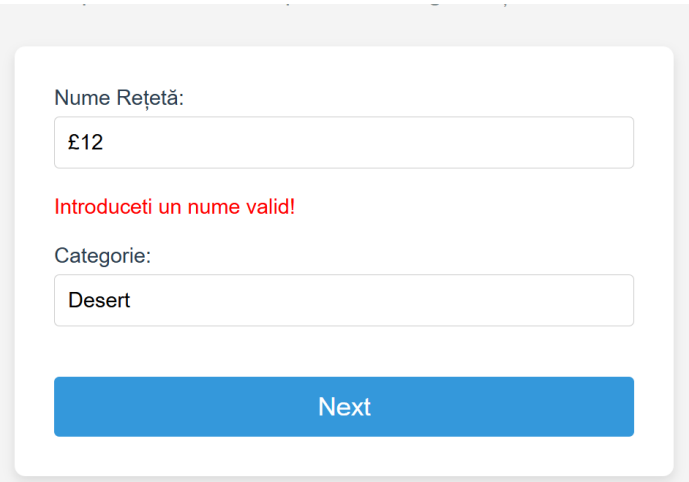
```
if (!nume.matches( regex: "^[a-zA-Z]+( [a-zA-Z]+)*$")) {
    model.addAttribute( attributeName: "errorNume",  attributeValue: "Introduceti un nume valid!");
    model.addAttribute( attributeName: "nume", nume);
    model.addAttribute( attributeName: "categorie", categorie);
    return "adauga-reteta-page";
}
```

Am adaugat atributele in model pentru a pastra valorile campurilor in momentul in care utilizatorul primeste mesajul de eroare.

In HTML mesajul de eroare este afisat astfel:

```
<label for="nume">Nume Retetă:</label>
<input id="nume" name="nume" placeholder="Ex: Lava Cake" th:value="${nume}">

<div style="...">
    <span th:if="${errorNume}" th:text="${errorNume}">Introduceti un nume valid!</span>
</div>
<br>
```

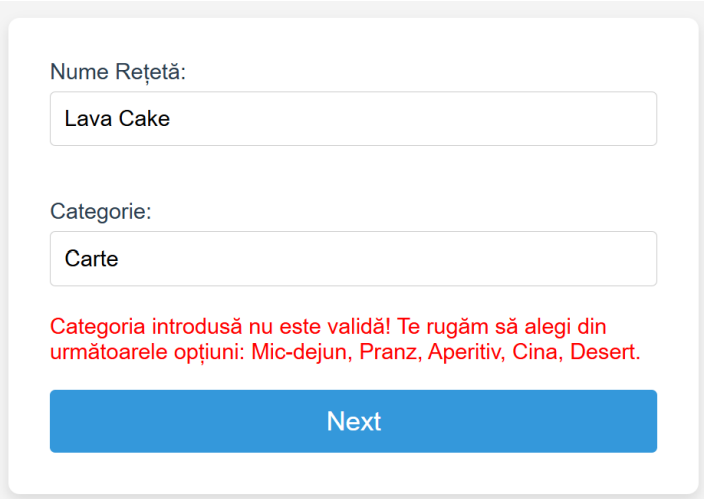
Iar in pagina mesajul va aparea sub campul care a fost intrdus gresit:



Pentru categorie verific daca datele introduce de utilizator fac parte din categoriile de mese predefinite:

```
List<String> categoriiPermise = List.of("Mic-dejun", "Pranz", "Aperitiv", "Cina", "Desert");

if (!categoriiPermise.contains(categorie)) {
    model.addAttribute( attributeName: "errorCategorie",  attributeValue: "Categoria introdusă nu este validă! Te rugăm să al
    model.addAttribute( attributeName: "categorie", categorie);
    model.addAttribute( attributeName: "nume", nume);
    return "adauga-reteta-page";
}
```

Iar mesajul de eroare in pagina va aparea astfel:



Asemanator am facut pentru fiecare camp al ingredientelor, etapelor si ustensilelor. Pentru datele introduce care reprezinta numere am verificat ca acestea sunt valide tot printr-o expresie regulate, iar apoi le am parsat ca int/ double in functie de fiecare.

```
if (!pretProdus.matches( regex: "^[0-9]+(\\.[0-9]+)?$")) {
    redirectAttributes.addFlashAttribute( attributeName: "errorPret",  attributeValue: "Pretul trebuie să fie un număr.");
    redirectAttributes.addFlashAttribute( attributeName: "nume", nume);
    redirectAttributes.addFlashAttribute( attributeName: "categorie", categorie);
    redirectAttributes.addFlashAttribute( attributeName: "cantitate", cantitate);
    redirectAttributes.addFlashAttribute( attributeName: "unitateMasura", unitateMasura);
    redirectAttributes.addFlashAttribute( attributeName: "pretProdus", pretProdus);
    return "redirect:/ingrediente?retetaId=" + retetaId;
}
double pretProdusValoare = Double.parseDouble(pretProdus);
```

5.2 Validare pentru modificarea si cautarea retetelor

Pentru validare le modificarea retetelor am utilizat aceleasi metode de validare ca pentru adaugare, deoarece campurile sunt identice.

Pentru cautare am facut validarea cu ajutorul unei expresii regulate care verifica daca datele introduse sunt de forma unei liste de produse separate prin virgula, care se pot scrie cu spatii intre ele.

```
@PostMapping("/cauta-reteta")
public String cautaReteta(@RequestParam String ingrediente, Model model, RedirectAttributes redirectAttributes) {

    String regex = "^(\\s*[a-zA-Z ]+\\s*)(,\\s*[a-zA-Z ]+\\s*)*$";

    if (!ingrediente.matches(regex)) {
        redirectAttributes.addFlashAttribute( attributeName: "errorLista",  attributeValue: "Introduceti ingredientele separate prin virgulă
        redirectAttributes.addFlashAttribute( attributeName: "ingrediente", ingrediente);
        return "redirect:/cauta-reteta";
    }
```

6. Testare

6.1 Testare Setters & Getters

Pentru testarea acestora am utilizat metoda uzuala de a da valori si a verifica daca rezultatul obtinut la apelarea metodei este egal cu cel dat de mine. Am facut aceste teste pentru fiecare clasa in parte.

```java
@Test
public void testGetterAndSetterForNume() {
    String numeTest = "Mancare de porc";
    reteta.setNume(numeTest);

    assertEquals(numeTest, reteta.getNume(), message: "Getter-ul pentru nume nu returnează valoarea
}

@Test
public void testGetterAndSetterForCategorie() {
    String categorieTest = "Fel principal";
    reteta.setCategorie(categorieTest);

    assertEquals(categorieTest, reteta.getCategorie(), message: "Getter-ul pentru categorie nu retu
}
```

```
Tests passed: 4 of 4 tests – 133 ms
RetetaTest (com.retete.retete_ 133 ms
    testGetterAndSetterForCos 118 ms
    testGetterAndSetterForNume 6 ms
    testGetterAndSetterForCateg 6 ms
    testGetterAndSetterForTimpF 3 ms

C:\Users\mirun\.jdks\openjdk-20.0.1\bin\java.exe ...

Process finished with exit code 0
```

Am facut identic si pentru celelalte clase.

6.2 Testare metode Service

Pentru testarea metodelor din service, am facut cate o clasa de test pentru fiecare in parte. Voi exemplifica testarea pentru clasa Ingredient.

Am creat un obiect de tip ingredient:

```java
@BeforeEach
public void setUp() {
    MockitoAnnotations.openMocks( testClass: this);
    ingredient = new Ingredient();
    ingredient.setId(1L);
    ingredient.setNume("Zahăr");
    ingredient.setCategorie("Dulciuri");
    ingredient.setCantitate(500.0);
    ingredient.setUnitateMasura("g");
    ingredient.setPretProdus(10.0);
    ingredient.setPretTotal(10.0);
}
```

- Test pentru functia de salvare

```
@Test
public void testSalveazaIngredient() {
    when(ingredientRepository.save(any(Ingredient.class))).thenReturn(ingredient);
    ingredientService.salveazaIngredient(ingredient);
    verify(ingredientRepository, times( wantedNumberOfInvocations: 1)).save(ingredient);
}
```

- Test pentru gasirea ingredientului dupa id-ul retetei

```
@Test
public void testGasesteIngredienteDupaRetetaId() {
    when(ingredientRepository.findByRetetaId(1L)).thenReturn(Collections.singletonList(ingredient));
    List<Ingredient> ingrediente = ingredientService.gasesteIngredienteDupaRetetaId(1L);
    assertNotNull(ingrediente);
    assertEquals( expected: 1, ingrediente.size());
    assertEquals(ingredient, ingrediente.get(0));
}
```

- Test gasirea ingredientului dupa id

```
@Test
public void testGasesteDupaId() {
    when(ingredientRepository.findById(1L)).thenReturn(Optional.of(ingredient));
    Ingredient foundIngredient = ingredientService.gasesteDupaId(1L);
    assertNotNull(foundIngredient);
    assertEquals(ingredient.getId(), foundIngredient.getId());
}
```

- Test pentru modificarea ingredientului

```
@Test
public void testUpdateIngredient() {
    when(ingredientRepository.findById(1L)).thenReturn(Optional.of(ingredient));
    when(ingredientRepository.save(any(Ingredient.class))).thenReturn(ingredient);

    ingredient.setNume("Miere");
    ingredientService.updateIngredient( id: 1L, ingredient);
    verify(ingredientRepository, times( wantedNumberOfInvocations: 1)).save(ingredient);
}
```

7. Concluzie

7.1 Imbunatatirea aplicatiei

Aici sunt 5 idei pentru imbunatatirea aplicatiei de retete culinare:

1. Adaugarea unui buton pentru fiecare reteta in pagina de cautare a retetelor care sa iti afiseze reteta.
2. In pagina de afisare a retetleor as pune doar titlurile si descrierea sumara a retetelor, iar apoi cand apesi pe un buton de arata reteta sa apara in detaliu.
3. As face pentru pagina de modifcare un stil mai ordonat, cu fiecare ingredient, etapa si ustensila mai restranse.
4. Filtru pentru categoria din care face parte reteta (Mic-dejuna, Pranz, etc).
5. Sugestii de reteta in functie de un budget.

7.2 Concluzie

Aplicatia web de retete culinare este foarte utila pentru oricine vrea sa incerce retete noi sau sa gaseasca idei pentru pranz sau cina. Ea iti permite sa cauti retete dupa ingrediente si sa vezi pasii pentru fiecare reteta. De asemenea, poti salva retetele tale preferate si le poti gasi usor cand ai nevoie de ele. Aplicatia este simplu de folosit si ajuta mult cand vrei sa gatesti ceva rapid sau sa inveti retete noi.

In concluzie, este un ajutor grozav pentru orice persoana care iubeste sa gateasca.